Dhara — Flash Flood Early Warning System

Full Project Documentation

Smart India Hackathon (SIH) — Problem Statement 192 Prototype: Hyper-local flash-flood risk monitoring for Himalayan hill states

---

1. Executive summary

Dhara is a web-based flash-flood early-warning prototype designed for settlements in hilly and Himalayan regions. It brings rainfall forecasts, modelled soil moisture, terrain/slope references, historical hazard frequency, satellite map context, field photographs, and an explainable risk score into one dashboard.

The system is intentionally built as a small, understandable hackathon architecture:

• **Frontend:** HTML, CSS, vanilla JavaScript, Leaflet, Chart.js.

• **Backend:** Python + Flask.

• **Live weather/soil source:** Open-Meteo forecast API.

• **Map:** OpenStreetMap for the normal map and Esri World Imagery for satellite context.

• **Storage:** JSON files plus image folders for the prototype.

• **Risk model:** transparent weighted scoring from 0–100.

• **Assistant:** rule-based prevention and preparedness chatbot using a local knowledge base.

This is a prototype and must not be presented as an operational emergency-warning system. Terrain and historical values in the bundled dataset are explicitly reference/prototype values and should be replaced with authoritative datasets before real-world deployment.

---

2. Core objectives

1. Show a clear risk level for monitored hill settlements.

2. Combine multiple factors instead of relying on rainfall alone.

3. Make the risk score explainable to a human operator.

4. Give an operator a map-first view with fast city selection.

5. Provide satellite context for terrain and drainage inspection.

6. Allow field teams to attach photographs for soil, terrain, rivers, land use, and flood evidence.

7. Provide practical preparedness guidance through a small assistant.

8. Keep the project easy to run locally and easy to deploy as one Flask service.

---

3. User experience

3.1 Main dashboard

The main screen contains:

• Dhara brand/header and live-data status.

• State filter and monitored-location watchlist.

• Current risk KPI cards.

• Interactive Leaflet map.

• Overview / Satellite map switch.

• Risk-score bar chart.

• Alert-band pie chart.

• Priority locations list.

• City intelligence drawer.

• Flash Flood Assist chat control.

The existing Dhara green/cream palette is deliberately retained. New charts use the same alert-band colours rather than introducing a new visual theme.

3.2 City intelligence

Selecting a city from the watchlist, map marker, or priority list opens its detail panel. The panel includes:

• State and district.

• Risk score and alert band.

• Plain-language risk description.

• Rainfall for the next 6 hours.

• Rainfall for the next 24 hours.

• Peak hourly rainfall intensity.

• Shallow soil moisture.

• Four risk-factor subscores.

• River basin.

• Slope class.

• Soil type.

• Elevation.

• Satellite-context action.

• Field evidence gallery.

• Field-photo upload controls.

• Data-source/update note.

3.3 Human-centred wording

The UI avoids overly technical labels where a short human phrase is clearer. Examples include:

• “Where attention is needed” instead of only “Risk by Location”.

• “Places worth checking first” for the priority list.

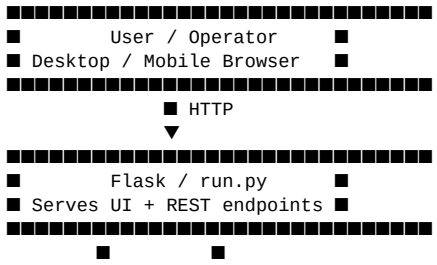
• “Real observations can sit beside the modelled numbers.” for field evidence.

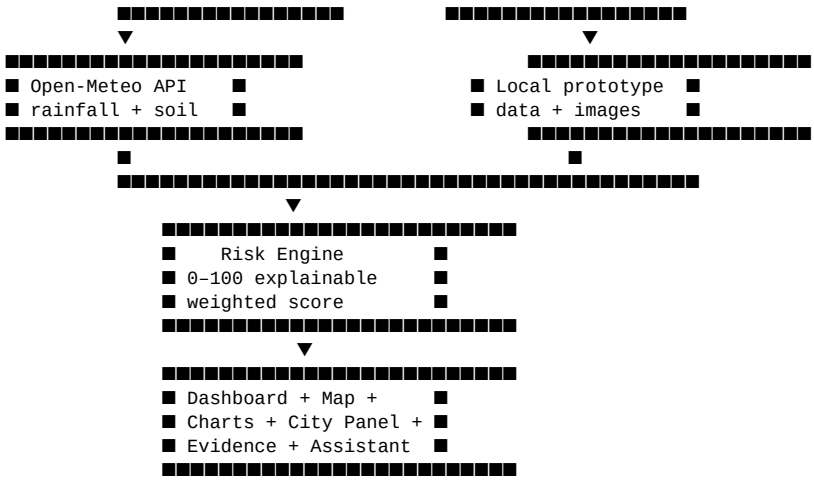
• “Highest forecast accumulation” for the rainfall KPI.

The purpose is to make the dashboard understandable during a stressful monitoring situation without removing the underlying technical information.

---

4. System architecture





Request flow

1. Browser loads `/`.

2. JavaScript initializes Leaflet.

3. Browser requests `/api/config`.

4. Browser requests `/api/risk-data`.

5. Flask loads the 25 monitored locations.

6. For each location, the backend calls `fetch_live_conditions()`.

7. Live values are passed to `compute_risk()`.

8. Results are returned as JSON.

9. The browser updates markers, KPIs, charts, and priority cards.

10. Selecting a location calls `/api/risk-data/<location_id>`.

11. The detail panel loads field evidence from `/api/evidence/<location_id>`.

12. Uploading a field photo sends multipart form data to the appropriate evidence endpoint.

---

5. Technology stack

Layer	Technology	Purpose
UI	HTML5	Page structure
Styling	CSS3	Dhara visual system and responsive layout
Client logic	Vanilla JavaScript	API calls, state, map and interaction
Maps	Leaflet 1.9.4	Interactive mapping
Normal basemap	OpenStreetMap tiles	Street/terrain context
Satellite basemap	Esri World Imagery	Satellite context
Charts	Chart.js 4.4.7	Bar and pie visualisations
Server	Flask 3.0.3	Web server and REST API
CORS	Flask-Cors 4.0.1	Cross-origin support if required later
HTTP	Requests 2.32.3	Open-Meteo requests
Config	python-dotenv 1.0.1	`.env` settings
Images	Pillow 12.3.0	Image validation/analysis
Production server	Gunicorn 23.0.0	Deployment process

---

6. Folder structure

```
flash-flood-prediction-sih192/
■
■■■ backend/
■   ■■■ __init__.py
■   ■■■ app.py
■   ■■■ chatbot.py
■   ■■■ config.py
■   ■■■ data_fetcher.py
■   ■■■ evidence.py
■   ■■■ risk_engine.py
■   ■■■ soil_sample.py
■
■■■ data/
■   ■■■ locations.json
■   ■■■ knowledge_base.json
■   ■■■ evidence/             # created/used for field-factor images
■   ■■■ soil_samples/         # soil sample image feature
■
■■■ docs/
■   ■■■ FULL_DOCUMENTATION.md
■   ■■■ ARCHITECTURE.md
■   ■■■ API.md
■   ■■■ DATA_SOURCES.md
■   ■■■ DEPLOYMENT.md
■   ■■■ FILE_GUIDE.md
■
■■■ frontend/
■   ■■■ index.html
■   ■■■ static/
■       ■■■ css/style.css
■       ■■■ js/
■           ■■■ app.js
■           ■■■ chatbot.js
■
■■■ .env.example
■■■ .gitignore
■■■ Procfile
■■■ README.md
■■■ requirements.txt
■■■ run.py
```

---

7. Data model

Location data

`data/locations.json` contains the monitored settlements. The project currently contains 25 locations across:

• Himachal Pradesh

• Jammu & Kashmir

• Ladakh

• Uttarakhand

Each location provides identifying information and prototype terrain/hazard references such as latitude, longitude, district, river basin, slope class, soil type, elevation and historical-event count.

Live data

The weather module requests hourly data for:

• precipitation

• soil moisture at 0–7 cm

• soil moisture at 7–28 cm

• temperature at 2 m

• weather code

The forecast request uses automatic local timezone handling and identifies the current hourly position instead of assuming a fixed array index.

---

8. Risk scoring model

Dhara uses a transparent weighted model rather than an opaque ML prediction.

Weights

Factor	Weight
Rainfall intensity and accumulation	45%
Soil saturation proxy	25%
Slope stability reference	20%
Historical hazard frequency	10%
Total	**100%**

Formula

```
Risk Score =
    0.45 × Rainfall Subscore
  + 0.25 × Soil Subscore
  + 0.20 × Slope Subscore
  + 0.10 × Historical Hazard Subscore
```

The final result is clamped to 0–100 and rounded to one decimal place.

Rainfall subscore

The rainfall component combines:

• worst-hour intensity

• next-6-hour accumulation

• next-24-hour accumulation

Approximate normalisation:

```
Intensity score = min(100, hourly_intensity / 20 × 100)
6h score        = min(100, rain_6h / 60 × 100)
24h score       = min(100, rain_24h / 150 × 100)

Rainfall subscore =
    0.50 × intensity score
  + 0.30 × 6h score
  + 0.20 × 24h score
```

Soil subscore

Open-Meteo soil moisture is treated as a saturation proxy:

```
Weighted soil moisture =
    0.60 × shallow moisture
  + 0.40 × 7–28 cm moisture

Soil subscore = clamp(weighted moisture / 0.45 × 100, 0, 100)
```

This is not a direct measurement of field water content at a specific site.

Slope subscore

The current prototype maps slope class to a simple score:

Slope class	Score
Low	10
Moderate	40
High	70
Very High	100

Historical hazard subscore

The historical event count is normalised against 13 events:

```
Historical subscore = min(100, events / 13 × 100)
```

Alert bands

Score	Band	Meaning
0–20	Safe	Routine monitoring
20–40	Watch	Conditions building
40–60	Warning	Prepare to move
60–80	Danger	High risk; evacuate exposed areas
80–100	Severe	Life-threatening; follow authorities

Important modelling limitation

The slope and historical components are prototype reference values. They are not a substitute for authoritative terrain, landslide susceptibility, river-gauge, drainage, or historical-event datasets. The risk score is therefore a **decision-support prototype**, not an official evacuation trigger.

---

9. Live weather/soil integration

The integration lives in `backend/data_fetcher.py`.

The backend sends an HTTP request to the Open-Meteo forecast endpoint with latitude, longitude, hourly variables, forecast days, past days, and automatic timezone.

A small in-memory cache prevents repeated upstream requests for the same rounded coordinate during the cache TTL.

If the upstream service is unreachable, the application fails softly and returns fallback values with:

```
source = open-meteo-unreachable
```

The UI then tells the operator that the live feed is unavailable instead of pretending the values are live.

---

10. Satellite map feature

The dashboard has two map modes:

Overview

OpenStreetMap tiles provide the standard geographic context.

Satellite

Esri World Imagery provides satellite imagery for visual inspection of terrain, settlement patterns, drainage and surrounding land cover.

This is **not Google Earth 3D** and should not be described as Google Earth. The current implementation uses an Esri tile service. A production implementation should review the applicable service terms, attribution requirements, quotas and caching rules.

When a city is selected, the “Open map” action switches to satellite mode and zooms toward the selected coordinates.

---

11. Field evidence / image feature

The field evidence system allows an operator to attach one latest image for each factor category per city:

1. Soil
2. Terrain / slope
3. River / drainage
4. Land use / vegetation
5. Flood evidence

Why this matters

Numerical weather models can miss a local observation. A field worker may see:

• exposed or saturated soil,

• a blocked drain,

• an overflowing stream,

• a fresh slope crack,

• unusual surface runoff,

• recent flood marks.

The prototype therefore places human observations beside the modelled risk values.

Upload flow

```
Choose city
   ↓
Choose evidence category
   ↓
Click “Add field photo”
   ↓
Select/capture JPG, PNG or WEBP
   ↓
POST /api/evidence/<location>/<category>
   ↓
Save latest image + metadata
   ↓
Refresh city evidence gallery
```

Images are stored under:

```
data/evidence/<location_id>/<category>/
```

The image metadata includes upload time and a simple visual description.

Honesty limitation

The current image analysis is a lightweight brightness/RGB heuristic. It is **not** a calibrated soil-moisture classifier or a scientific terrain detector. The code deliberately labels these values as descriptive evidence. A future version can replace it with a trained model using labelled field imagery.

---

12. Soil sample feature

The older soil-specific upload route remains available separately from the multi-factor evidence gallery.

Endpoints:

```
GET  /api/soil-sample/<location_id>
POST /api/soil-sample/<location_id>
GET  /api/soil-sample/<location_id>/image
```

It stores one latest soil sample per location under `data/soil_samples/`.

The image heuristic produces a qualitative moisture hint such as dark/moderate/dry. This must not be described as a sensor reading.

---

13. Charts

Risk-score bar chart

The dashboard displays the highest-risk locations in a horizontal bar chart. Each bar retains the alert-band colour of that location.

Purpose:

• compare locations quickly,

• identify outliers,

• support prioritisation.

Alert-band pie chart

The dashboard also displays the share of all monitored locations in:

• Safe

• Watch

• Warning

• Danger

• Severe

Purpose:

• show the overall health of the watchlist at a glance,

• communicate whether risk is concentrated or widespread.

Chart.js is loaded from a CDN. If a deployment needs to operate fully offline, the Chart.js asset should be self-hosted in the project.

---

14. Chatbot / Flash Flood Assist

The assistant is a local rule-based system. It does not require an LLM API key.

Main files

```
frontend/static/js/chatbot.js
backend/chatbot.py
data/knowledge_base.json
```

The browser sends:

```
POST /api/chatbot
Content-Type: application/json
```

with:

```
{"message":"What should I do during a flash flood?"}
```

The backend returns a rule-based answer from the local knowledge base.

Chat UI behaviour

The assistant is intentionally opened as a separate floating panel so it does not cover the city intelligence drawer. It can be closed with:

• the `×` button,

• the Escape key,

• selecting another city.

The close control explicitly manages the panel's `hidden`, CSS class, and ARIA state.

---

15. API reference

`GET /api/health`

Health check.

Example response:

```
{"status":"ok","locations_loaded":25}
```

`GET /api/config`

Returns public frontend configuration.

Includes:

• refresh interval,

• optional Mapbox token,

• satellite tile URL,

• satellite attribution.

Private API keys are not returned.

`GET /api/locations`

Returns the configured monitored locations.

`GET /api/risk-data`

Calculates current risk for every configured location.

The result is sorted from highest to lowest risk.

`GET /api/risk-data/<location_id>`

Returns detailed risk information for one location.

`POST /api/chatbot`

Returns a rule-based preparedness answer.

`GET /api/chatbot/greeting`

Returns the assistant's greeting.

`GET /api/evidence/<location_id>`

Returns all uploaded factor photos for a city.

`POST /api/evidence/<location_id>/<category>`

Accepts a multipart form field named `photo`.

Supported categories:

```
soil
terrain
river
landuse
flood_evidence
```

`GET /api/evidence/<location_id>/<category>/image`

Returns the latest stored evidence image.

---

16. Installation on Windows

Open Command Prompt in the extracted project directory.

Create virtual environment

```
python -m venv venv
```

Activate

```
venv\\Scripts\\activate
```

Install dependencies

```
pip install -r requirements.txt
```

Run

```
python run.py
```

Open:

```
http://localhost:5000
```

If Pillow installation fails

Use a current Pillow release compatible with the installed Python version. The supplied project pins Pillow 12.3.0 for the current Python 3.14 environment.

---

17. Environment configuration

Copy:

```
.env.example → .env
```

The project works without API keys for its default Open-Meteo and map configuration.

Important settings include:

```
PORT=5000
FLASK_ENV=development
REFRESH_INTERVAL_SECONDS=300
CACHE_TTL_SECONDS=180
```

Optional values include:

```
OPENWEATHER_API_KEY=
MAPBOX_ACCESS_TOKEN=
```

Never commit private secrets to GitHub.

---

18. Deployment

Recommended prototype deployment

A GitHub repository connected to a Flask-compatible hosting service such as Render is a simple route for an SIH demo.

Typical production start command:

```
gunicorn run:app
```

The repository already includes a `Procfile` containing the Gunicorn command.

Deployment considerations

The prototype currently writes uploaded images to the local filesystem. Many cloud services use ephemeral filesystems, so uploaded field images may disappear after a redeploy/restart.

For production, move evidence storage to an object-storage service such as S3-compatible storage and store metadata in a database.

---

19. Production upgrade path

Data

Replace prototype references with:

• authoritative rain gauges,

• river/stream gauges,

• local IoT telemetry,

• official terrain/elevation products,

• landslide susceptibility layers,

• historical flood inventories,

• satellite-derived land-cover and soil indicators.

Model

Replace the weighted score with a validated model trained on historical events.

Possible architecture:

```
Weather + gauges + soil + DEM + land cover + river data
                          ↓
                  Feature engineering
                          ↓
                Time-aware ML model
                          ↓
             Probability of flash flood
                          ↓
            Calibration + thresholds
                          ↓
              Explainable alert
```

Validation should use geographically and temporally separated data to reduce leakage.

Imagery

A future version can add:

• Sentinel-derived indices,

• DEM/slope maps,

• change detection,

• river-width changes,

• cloud-aware satellite composites.

Infrastructure

For production:

• PostgreSQL/PostGIS for geospatial data.

• Redis for caching.

• Object storage for photos.

• Background workers for scheduled ingestion.

• Authentication and role-based access.

• Audit logs.

• Monitoring and alert delivery.

• SMS/app notification integrations.

---

20. Security and operational notes

1. Do not expose private API keys through `/api/config`.

2. Validate uploaded file extensions and image content.

3. Limit image size.

4. Use generated filenames rather than trusting user filenames.

5. Add authentication before allowing public uploads.

6. Add rate limiting to upload and chatbot endpoints.

7. Store production evidence outside the web server filesystem.

8. Keep an audit trail for changes to risk thresholds and location data.

9. Clearly label modelled values versus field observations.

10. Never present the prototype score as an official evacuation order.

---

21. Testing checklist

Local smoke test

• [ ] `python run.py` starts without traceback.

• [ ] `/api/health` returns `status: ok`.

• [ ] Dashboard loads.

• [ ] Watchlist contains locations.

• [ ] Map loads.

• [ ] Satellite mode switches correctly.

• [ ] Selecting a marker opens city intelligence.

• [ ] Risk bar chart renders.

• [ ] Alert pie chart renders.

• [ ] Chat assistant opens.

• [ ] Chat assistant closes using `×`.

• [ ] Chat assistant closes using Escape.

• [ ] Evidence category selector works.

• [ ] Soil image upload works.

• [ ] Terrain image upload works.

• [ ] River image upload works.

• [ ] Land-use image upload works.

• [ ] Flood-evidence upload works.

• [ ] Uploaded image appears in the city panel.

Network failure test

Temporarily disable internet access and verify that the UI reports that the live feed is unavailable rather than claiming that fallback values are live.

Deployment test

• [ ] Build completes.

• [ ] Gunicorn starts.

• [ ] Root route loads.

• [ ] API routes work.

• [ ] CDN assets load.

• [ ] Satellite imagery loads.

• [ ] Uploaded evidence storage behaviour is understood.

---

22. Judge demonstration flow

A clean 3–5 minute demonstration can follow this sequence:

Step 1 — Explain the problem

Flash floods in hilly regions can develop quickly, so a useful warning system should combine rainfall with local susceptibility rather than showing weather alone.

Step 2 — Show the overview

Point out the live status, monitored locations, risk distribution, and priority list.

Step 3 — Select a high-risk city

Show how the map zooms to the location and the city intelligence panel explains the score.

Step 4 — Explain the score

Show the four factor bars and explain the 45/25/20/10 weighting.

Step 5 — Show satellite context

Switch to satellite imagery and explain that it helps an operator visually inspect terrain, drainage and settlement surroundings.

Step 6 — Show field evidence

Upload a soil or terrain photograph and demonstrate that a human observation can be attached to the city.

Step 7 — Show charts

Use the bar chart to compare risk and the pie chart to communicate the overall alert mix.

Step 8 — Ask the assistant

Ask a preparedness question and show the rule-based local response.

Step 9 — Be transparent

Explain which inputs are live, which values are prototype references, and how the system would be upgraded with authoritative data and validated ML.

---

23. What is live vs prototype

Component	Current status
Rainfall forecast	Live Open-Meteo request
Modelled soil moisture	Live Open-Meteo request
Temperature/weather code	Live Open-Meteo request
Map	Live online tiles
Satellite imagery	Live online Esri imagery
Risk calculation	Live calculation from current inputs
Slope reference	Prototype/static
Historical hazard frequency	Prototype/static
Soil field photo	User-provided evidence
Other factor photos	User-provided evidence
Image analysis	Simple heuristic, not calibrated ML
Chatbot	Local rule-based knowledge base
Database	Not used; JSON/filesystem prototype
IoT sensors	Not connected in this ZIP
Official evacuation integration	Not connected

---

24. Key limitations

Dhara is an SIH prototype. It demonstrates the architecture and user experience of a multi-source early-warning platform, but it does not establish operational flood probability or official evacuation thresholds.

The largest technical upgrades needed before real deployment are authoritative geospatial/historical data, local telemetry, validated probabilistic modelling, robust storage, authentication, monitoring, and formal safety/validation processes.

---

25. Final project summary

Dhara demonstrates a practical end-to-end pattern for a multi-source flash-flood decision-support system:

```
Live weather/soil
       +
Terrain references
       +
Historical hazard references
       +
Satellite context
       +
Human field evidence
       ↓
Explainable risk score
       ↓
Map + charts + city intelligence
       ↓
Preparedness assistant
       ↓
Human decision / official response workflow
```

The strongest part of the prototype is its transparency: every major displayed risk factor can be traced to a named input, and human field observations are explicitly separated from modelled values.

Prototype status: suitable for demonstration and further development; not for autonomous emergency decisions.